

Ohanyere Francis

Platform Engineer

+234 813 432 0174 | ohanyerefrancis395@gmail.com | github.com/ohanyere | linkedin.com/in/ohanyere-francis

Professional Summary

Platform Engineer building **self-service developer platforms** and **Kubernetes control planes** that **reduce the manual work required to create services, provision infrastructure, deploy software, and operate applications**. Builds Platform APIs, reusable CI/CD and GitOps workflows, declarative infrastructure, security guardrails, and observability so developers can **work through approved platform paths without managing the underlying cloud and Kubernetes complexity**. Reduced new-service onboarding **from 2 weeks to under 15 minutes** and environment provisioning **from 15 minutes to under 30 seconds**.

Professional Experience

Platform Engineering Instructor

ICT Slides

2022 – Present

Anambra State, Nigeria

- Built and evolved **self-service platform capabilities** that let engineers create services, request infrastructure, deploy applications, and operate workloads without routine platform-team setup; common platform requirements such as ownership, access, security, delivery configuration, and observability were applied automatically as part of those workflows.
- Built **self-service infrastructure provisioning** that let engineers request approved cloud resources through **version-controlled workflows instead of manual cloud-console changes or infrastructure tickets**. The platform managed provisioning, ownership, access controls, encryption and network defaults, cost-allocation metadata, drift correction, updates, and lifecycle cleanup.
- Standardized software delivery through **reusable CI/CD and GitOps workflows covering build and validation, versioned releases, environment promotion, progressive delivery, health-based rollout decisions, and rollback**, giving engineers one maintained path from source code to production rather than separate delivery processes for every application; feature flags allowed release decisions to be separated from deployment.
- Embedded **security and governance by default** so teams inherited **image scanning, signed artifacts, policy validation, secrets handling, least-privilege access, and workload security controls** without having to reproduce those protections independently for every service.
- Developed **Platform APIs and Kubernetes controllers in Go** that automated **environment and tenant lifecycle, access, resource limits, networking, security policy, and reconciliation**, replacing repeated cluster administration with declarative platform-managed control loops.
- Built **observability, reliability, and recovery capabilities** that gave engineers both the operational signals needed to **diagnose failures and defined paths to recover services when they failed**. Metrics, logs, traces, SLOs, alerting, deployment health, backups, restore validation, and disaster-recovery workflows covered the service lifecycle beyond deployment.
- Used the platforms and systems we built to **teach and mentor engineers through production-style application, infrastructure, delivery, security, and reliability problems**, connecting individual technologies to the engineering decisions behind operating a complete platform.

Selected Platform Engineering Work

Internal Developer Platform

2024

nexus-idp-web.vercel.app

- Go · TypeScript · Kubernetes · AWS · GitHub Actions · ArgoCD · Crossplane · Argo Rollouts · Istio · Prometheus · Vault · Kyverno*
- Golden Path service creation lets **developers choose an approved service template and receive a repository with CI/CD, GitOps manifests, Helm configuration, environment overlays, rollout strategy, ownership, security defaults, and observability already connected**. Nexus generates the repository through the Platform API, reducing new-service onboarding **from 2 weeks to under 15 minutes** without requiring teams to assemble delivery infrastructure themselves.
 - Software delivery is standardized so **reusable CI/CD workflows turn source code into tested, scanned, signed, and versioned release artifacts, while GitOps promotes the same approved release through development, staging, and production without rebuilding it for each environment**. Nexus exposes deployment history, approval gates, GitHub Actions status, ArgoCD sync health, canary or blue-green rollout progress, and rollback through a single release workflow.
 - Self-service infrastructure lets **developers request approved cloud resources while the platform turns each request into a version-controlled Crossplane-managed AWS resource with ownership, access, security, cost, drift, and lifecycle controls applied automatically**. The Platform API creates the GitHub change, ArgoCD reconciles the Crossplane claim, and Crossplane provisions databases, storage, queues, and other resources without routine infrastructure tickets or manual cloud-console provisioning.

- Progressive delivery uses Argo Rollouts, Istio, and Prometheus so **new versions are exposed gradually, promoted only while service health remains within acceptable limits, and automatically rolled back when a release becomes unhealthy**. Feature flags provide a separate control for gradually exposing functionality without tying every product-release decision to another deployment.
- Service operations bring **ownership, deployed versions, SLOs, metrics, logs, traces, alerts, ArgoCD state, policy violations, infrastructure drift, resource utilisation, and cost into one operational view**, helping engineers understand what is running, what changed, and what requires action. Automated backups, defined RPO/RTO targets, and restore validation provide a tested recovery path when rollback is not enough, while consolidated operational context reduced **MTTR from 2 hours to 1 hour**.

Kubernetes Tenant Platform – Multi-Tenant Kubernetes Control Plane

2025

Live Demo

Go · TypeScript · Kubernetes · controller-runtime · Kubebuilder · ArgoCD · GitOps · PostgreSQL

- Self-service tenancy lets **teams request a complete Kubernetes environment through one Tenant contract instead of creating namespaces, quotas, RBAC, network rules, and workload-security settings themselves**. Developers declare team, environment, capacity, connectivity, and access intent while the control plane turns those profiles into a managed runtime boundary and reports whether each platform capability is ready.
- Desired state is separated deliberately so **Git owns the requested Tenant configuration, Kubernetes represents the current runtime state, and PostgreSQL-backed platform events preserve the durable history of what happened**. Governed changes move through GitHub branches and pull requests, ArgoCD reconciles the Tenant resource, and the operator continuously translates that high-level intent into the Kubernetes resources required by the environment.
- Reconciliation is designed to make the platform a safe long-running owner of tenant environments: **missing managed resources are recreated, platform-owned drift is corrected, unrelated metadata is preserved, and resources belonging to another owner are never silently adopted**. Ownership conflicts stop reconciliation safely, while healthy environments converge to a **zero-write steady state** instead of continuously generating unnecessary Kubernetes API updates.
- Tenant guardrails keep **team permissions bounded while workload security, capacity, and network boundaries remain centrally controlled by the platform**. Group-based access profiles provide least-privilege developer permissions, Pod Security Admission applies restricted workload defaults, and platform-owned NetworkPolicies enforce default-deny and approved connectivity without giving application teams control of the tenant boundary itself.
- Tenant lifecycle and failure handling are treated as control-plane responsibilities: **readiness is reported independently for namespace, capacity, access, and networking, destructive deletion requires approval bound to the exact Tenant UID, and retained resources are never automatically adopted by a replacement Tenant**. Fresh ownership checks, UID delete preconditions, namespace-last cleanup, durable audit events, and Lease-based leader election make failure, deletion, and controller recovery explicit parts of the platform design rather than afterthoughts.

Technical Skills

Languages & Platform Development: Go · TypeScript · Platform APIs · controller-runtime · Kubebuilder

Cloud & Infrastructure: AWS · GCP · Kubernetes · Terraform · Crossplane · Helm · Docker · Linux

Delivery & GitOps: GitHub Actions · ArgoCD · Argo Rollouts · Istio

Observability & Security: Prometheus · Grafana · OpenTelemetry · Vault · Kyverno · Trivy · Cosign

Education

Chukwuemeka Odumegwu Ojukwu University

B.Sc. Education and Computer Science Candidate

ALX Africa

2022 – 2023

Software Engineering Fellowship

- * Software engineering foundations across **C, Python, Data Structures & Algorithms, Linux/Unix, Shell Scripting, Git, SQL/Databases, Networking, REST APIs, and Backend Development**.